

89. Capturing Groups

Pattern ke kisi hisse ko parentheses `(...)` me daalna **capturing group** kehata hai. Iske 2 asar:

1. Match ka wo hissa result array me **alag item** ke roop me milta hai.
2. Parentheses ke baad quantifier lagao to wo **poore group par** lagta hai.

Examples

gogogo: `(go+)` = `(g)` ke baad `(o)` kai baar `(gooooo)`. Parentheses characters ko group karte hain:
`(go)+` = `(go)`, `(gogo)`, `(gogogo)`...

```
js
'Gogogo now!'.match(/(go)+/ig);    // "Gogogo"
```

Domain: `(mail.com)`, `(users.mail.com)`... (words, har ek ke baad dot, aakhri ko chhodkar) =
`(\w+\.)+\w+`

```
js
"site.com my.site.com".match(/(\w+\.)+\w+/g);    // site.com,my.site.com
```

Hyphen wale domain `(my-site.com)` ke liye `(\w)` ki jagah `([\w-])`: `([\w-]+\.)+\w+`.

Email: `(name@domain)`, name me hyphen aur dots allowed = `([-.\w]+)`

```
js
let regexp = /[-.\w]+@([\w-]+\.)+[\w-]+/g;
"my@mail.com @ his@site.com.uk".match(regexp);    // my@mail.com, his@site.com.uk
```

Perfect nahi par aksar chalta hai. Email ka asli pakka check sirf mail bhej kar hi ho sakta hai.

Match me parentheses ka content

Parentheses **left se right number** hote hain. `(str.match(regexp))` (bina `(g)` ke) array deta hai:

- Index `(0)`: poora match
- Index `(1)`: pehle parentheses ka content
- Index `(2)`: doosre ka...

```
js
```

```
let tag = '<h1>Hello, world!</h1>'.match(/<(.*?)>/);
tag[0];    // <h1>
tag[1];    // h1
```

Nested groups

Parentheses nested ho sakte hain. Numbering phir bhi **opening paren** ke hisaab se left se right. `<((([a-z]+)\s*([>]*))>)` on `<span class="my">`:

```
js

let result = '<span class="my">'.match(/<((([a-z]+)\s*([>]*))>)/);
result[0];    // <span class="my">
result[1];    // span class="my"    (poora tag content)
result[2];    // span                (tag name)
result[3];    // class="my"          (attributes)
```

Optional groups

Group optional ho aur match me na ho (jaise `(...)?`), tab bhi result array me uski jagah rehti hai, value `undefined`.

```
js

let match = 'a'.match(/a(z)?(c)?/);
match.length;    // 3
match[1];        // undefined
match[2];        // undefined

'ac'.match(/a(z)?(c)?/);    // ["ac", undefined, "c"]
```

Array ki length hamesha fix (3).

`matchAll`: `g` ke saath groups

! `matchAll` naya method hai, purane browsers me polyfill chahiye.

`g` flag ke saath `match` groups ka content **nahi** deta, sirf matches ki array. Groups chahiye to `str.matchAll(regexp)`. 3 farak (`match` se):

1. Array nahi, **iterable object** deta hai.
2. `g` ho to bhi har match **groups ke saath array** hota hai (bina `g` ke `match` jaisa format).
3. Koi match na ho to `null` nahi, **khaali iterable**.

```
js
```

```
let results = '<h1> <h2>'.matchAll(/<(.*?)>/gi);

results[0]; // undefined (ye array nahi hai)
results = Array.from(results); // array banao
results[0]; // <h1>,h1
results[1]; // <h2>,h2
```

Loop me `Array.from` ki zarurat nahi:

```
js

for (let result of '<h1> <h2>'.matchAll(/<(.*?)>/gi)) { ... }
let [tag1, tag2] = '<h1> <h2>'.matchAll(/<(.*?)>/gi); // destructuring
```

Har match me `index` aur `input` bhi hote hain.

Iterable kyun? Optimization. `matchAll` call karte hi search **nahi** karta. Search tab hoti hai jab iterate karo. Maano 100 matches hain par `for..of` me 5 milne ke baad `break` kar diya, to baaki 95 dhundhne me time nahi lagta.

Named groups

Numbers yaad rakhna mushkil hota hai, isliye parentheses ko naam do: opening paren ke turant baad `?<name>`.

```
js

let dateRegex = /(?!<year>[0-9]{4})-(?!<month>[0-9]{2})-(?!<day>[0-9]{2})/;
let groups = "2019-04-30".match(dateRegex).groups;

groups.year; // 2019
groups.month; // 04
groups.day; // 30
```

Groups match ki `.groups` property me hote hain. Saari dates ke liye `g` flag aur `matchAll`:

```
js

for (let result of "2019-10-30 2020-01-01".matchAll(dateRegex)) {
  let {year, month, day} = result.groups;
  alert(`${day}.${month}.${year}`); // 30.10.2019, phir 01.01.2020
}
```

Replacement me groups

`str.replace` ke replacement string me `$n` (n = group number) ya named ke liye `<name>`:

```
js
"John Bull".replace(/(\w+) (\w+)/, '$2, $1');    // Bull, John

let regexp = /(<year>[0-9]{4})-(<month>[0-9]{2})-(<day>[0-9]{2})/g;
"2019-10-30, 2020-01-01".replace(regexp, '$<day>.$<month>.$<year>');
// 30.10.2019, 01.01.2020
```

Non-capturing groups: `(?:)`

Kabhi quantifier sahi lagane ke liye parentheses chahiye par unka content result me nahi chahiye. Shuruaat me `(?:)` lagao:

```
js
let result = "Gogogo John!".match(/(?:go)+ (\w+)/i);

result[0];    // Gogogo John (poora match)
result[1];    // John
result.length; // 2 (aur koi item nahi)
```

Aise group ko replacement string me reference bhi nahi kar sakte.

Summary

- Parentheses regexp ke hisse ko group karte hain taaki quantifier poore par lage.
- Groups left se right number hote hain, aur `(?<name>...)` se naam de sakte hain.
- Content result me: `str.match` sirf bina `g` ke groups deta hai, `str.matchAll` hamesha deta hai. Unnamed ho to number se, named ho to `groups` property se bhi.
- Replacement me `$n` ya `<name>`.
- `(?:)` se group numbering se bahar (quantifier lagana ho par result me nahi chahiye).

Tasks (short)

- **MAC-address:** `/^[0-9a-f]{2}(:[0-9a-f]{2}){5}$/i`
- **Color** `#abc` ya `#abcdef`: `/#[a-f0-9]{3}{1,2}\b/gi` (`\b` taaki `#abcd` na match ho)
- **Saare decimal numbers (negative ke saath):** `/-?\d+(\.\d+)?/g`
- **Expression parse** `(1.2 * 3.4)`: `/((-?\d+(?:\.\d+)?)\s*([-\+*/])\s*(-?\d+(?:\.\d+)?))/`. Decimal parts ke groups `(?:)` se nikaale, aur `result.shift()` se poora match hataya, to `[number, operator, number]` bachta hai. Ya named groups `(a)`, `operator`, `(b)`.

